

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read images
img1 = cv2.imread("j1.jpg")
img2 = cv2.imread("j2.jpg")

# Convert to grayscale
gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)

# Add noise to second image
noise = np.random.normal(0, 25, gray2.shape).astype(np.float32)
noisy_img2 = cv2.add(gray2, noise)

# Remove noise using Gaussian blur
filtered_img2 = cv2.GaussianBlur(noisy_img2, (5, 5), 0)

# SIFT feature detection
sift = cv2.SIFT_create()

kp1, des1 = sift.detectAndCompute(gray1, None)
kp2, des2 = sift.detectAndCompute(filtered_img2, None)

# Feature matching
bf = cv2.BFMatcher()

# Check if descriptors are found for both images
if des1 is None or des2 is None or len(des1) == 0 or len(des2) == 0:
    print("Error: Not enough descriptors found in one of the images")
    good_matches = [] # Initialize good_matches to an empty list
else:
    matches = bf.knnMatch(des1, des2, k=2)

    # Ratio test
    good_matches = []
```

```
    for m, n in matches:
        if m.distance < 0.7 * n.distance:
            good_matches.append(m)

# Draw matches
result = cv2.drawMatches(
    gray1, kp1,
    filtered_img2, kp2,
    good_matches, None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

# Display
plt.figure(figsize=(12, 6))
plt.imshow(result, cmap="gray")
plt.title("Feature Matching After Noise Removal")
plt.axis("off")
plt.show()

print("Number of good matches:", len(good_matches))
```

```
-----
error                                Traceback (most recent call last)
/tmp/ipykernel_3182/2061331003.py in <cell line: 0>()
    32     good_matches = [] # Initialize good_matches to an empty list
    33 else:
--> 34     matches = bf.knnMatch(des1, des2, k=2)
    35
    36     # Ratio test

error: OpenCV(5.0.0) /io/opencv/modules/features/src/matchers.cpp:998: error: (-215:Assertion failed)
trainDescCollection[iIdx].rows < IMGIDX_ONE in function 'knnMatchImpl'
```

Next steps:

[Explain error](#)